

# python-notebook

February 26, 2024

```
[ ]: #print(2*3)
      #print (5/2)
      y = 2
      # 2*3 # _ stores the result of the previous computations
      # _+y
      # a = 4
      # b = 5
      # a + b
      # _ + y
```

```
[ ]: """
      Variable: Python variables are simply containers for storing data values_
      ↪[Memory location]
      Function: A function is a block of code which only runs when it is called and_
      ↪does some specific work
      """

      #### Python is a case sensitive language####
      name = 'my name is palash'
      Name = "my name is xxx"
      print(Name[0])
      NAME = "mm"
      print(name)
      print(Name)
      print(10*"pala")
      print(r"c:\abc\newfolder")
      print(Name[3:])
      print(len(Name))
```

```
[ ]: # A few common Datatypes:
      """
          Numeric Types:
              int: Represents integers (whole numbers).
              float: Represents floating-point numbers (numbers with a decimal point).
              complex: Represents complex numbers (numbers with a real and imaginary_
              ↪part).
```

```

Sequence Types:
    str: Represents strings (sequences of characters).
    list: Represents lists (ordered, mutable sequences).
    tuple: Represents tuples (ordered, immutable sequences).
Mapping Type:
    dict: Represents dictionaries (key-value mappings).
Set Types:
    set: Represents sets (unordered, mutable collections of unique
    ↪elements).
    frozenset: Represents frozensets (unordered, immutable collections of
    ↪unique elements).
Boolean Type:
    bool: Represents Boolean values (True or False).
None Type:
    NoneType: Represents the absence of a value or a null value. It is
    ↪often used to indicate the absence of a
    return value from a function.
Custom Types:
    You can define your own custom data types using classes.
"""
# x = 42          # int
# y = 3.14        # float
# z = 2 + 3j      # complex
# z1 = 4 + 1j
# z+z1

# is_true = True  # bool
# is_false = False # bool

x = None # NoneType
print (x)
type(x)

```

```

[ ]: #Boolean Variables and Logic:
is_python_fun = True
is_learning = True
can_program = is_python_fun and is_learning
print("Can I program?", can_program)

```

```

[ ]: ###dynamic data type #####
a = True
type(a) # type function helps us to check the datatype of a variable

```

```

[ ]: ## Variable Declaration and Assignment
name = "Alice"
age = 25

```

```

height = 5.8
is_student = True
print(name, age, height, is_student)
a = b = c = 677 # Multi assignment is also possible
b

```

```

[ ]: #User Input and Variables:
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello,", name)
print("Age:", age)

```

```

[ ]: #Arithmetic Operations with Variables:
x = 10
y = 5
sum_result = x + y
difference = x - y
product = x * y
print("Sum:", sum_result)
print("Difference:", difference)
print("Product:", product)

```

```

[ ]: # x = 5
# y = 10

# # To take inputs from the user
# x = input('Enter value of x: ')
# y = input('Enter value of y: ')

# # create a temporary variable and swap the values
# temp = x
# x = y
# y = temp

# print('The value of x after swapping: ', x)
# print('The value of y after swapping: ', y)
##### Without using third variable_
↪#####

# a = 5
# b = 10

# a = a + b
# b = a - b
# a = a - b

# print("After swapping:")
# print("a =", a)

```

```
# print("b =", b)
#####
"""
This XOR-based swapping technique works because the XOR operation is both
    ↪ commutative and associative, and it takes
advantage of the fact that XORing the same value twice cancels out the effect
    ↪ and returns the original value.
 $a \wedge (b \wedge c) = (a \wedge b) \wedge c = (a \wedge c) \wedge b$ 
a  b = b  a
"""
a = 5
b = 10

a = a ^ b
b = a ^ b
a = a ^ b

print("After swapping:")
print("a =", a)
print("b =", b)
```

```
[ ]: """
The bool() function returns the boolean value of a specified object.

The object will always return True, unless:

The object is empty
The object is False
The object is 0
The object is None

In Python, any non-empty string is treated as a true value in a Boolean context.
"""

# Input data from the user
name = input("Enter your name: ")
age = int(input("Enter your age: "))
height_feet = float(input("Enter your height in feet: "))
is_student = bool(input("Are you a student? Write Yes or leave it blank"))

# Calculate birth year
current_year = 2023
birth_year = current_year - age

# Convert height to centimeters
height_cm = height_feet * 30.48

# Display the results
```

```
print("Name:", name)
print("Birth Year:", birth_year)
print("Height in centimeters:", height_cm)
print("Student:", is_student)
```

```
[ ]: import math #imported math package, math.pi is an object

# Prompt the user for the radius
radius = float(input("Enter the radius of the circle: "))

# Calculate the area of the circle
area = math.pi * (radius ** 2)

# Display the result
print("The area of the circle with radius", radius, "is:", area)
```

```
[ ]: # Prompt the user for the temperature in Celsius
celsius = float(input("Enter temperature in Celsius: "))

# Convert Celsius to Fahrenheit
fahrenheit = (celsius * 9/5) + 32

# Display the result
print("Temperature in Fahrenheit:", fahrenheit)
```

```
[ ]: # Calculate the Hypotenuse of a Right Triangle
import math

# Input the lengths of the two shorter sides of a right triangle
side1 = float(input("Enter the length of the first side: "))
side2 = float(input("Enter the length of the second side: "))

# Calculate the length of the hypotenuse
hypotenuse = math.sqrt(side1**2 + side2**2)

# Display the length of the hypotenuse
print("The length of the hypotenuse is:", hypotenuse)
```

```
[ ]: t=89
#help(t) # this gives all information
dir(t) # this will give you information about only methods
```

```
[ ]: """
The id() function returns a unique id for the specified object.

All objects in Python has its own unique id.
```

The id is assigned to the object when it is created.

The id is the object's memory address, and will be different for each time you run the program.

If you are creating different variable with same value, python uses the existing storage location to avoid more memory utilization. But the range is [-5, 256]

```
"""
###memory management###
x = -6
y = -6
print ("both are same "+ str(id(x))+ ' ' + str(id(y)))
# y = 6
# print ("both are not same "+ str(id(x))+ " " + str(id(y)))
```

```
[ ]: ### Printing a value with our own message###

Num = 50 #int
print("This is a number", Num)
Num1 = '53'
print('This is a number', Num1)
print(Num+int(Num1)) #type casting can help in changing the datatypes, Here +
    ↳ is used as addition operator
print(str(Num)+Num1) #type casting. Here + operator is used as concatenation
```

```
[ ]: print('palash\notes') # \n means a new line
print(r"palash\notes")
```

```
[ ]: print("class doesn't have offline students")
print('class doesn\'t have offline students')
print('"class doesn\'t have offline students"')
```

```
[ ]: ## deleting a variable###
w=7
print(w)
del w # del keyword
print(w)
#w = w+3
print('hi')
```

```
[ ]: """
F-string provides a way to embed expressions inside string literals, using a
    ↳ minimal syntax.
"""
```

```
####printing values in between a string####
Name = "Sachin"
Age = 35
print("My name is:" + Name) #### Instead of ',' you can use '+'
print(f'My name is: {Name}, and my age is {Age}') # printing using f-string
y = f'My name is: {Name}, and my age is {Age}'
print(y)
print(f'My name is: {Name}, and my age is {Age+Age}')
```

```
[ ]: """
str.format() is one of the string formatting methods in python which allows
↳ multiple substitutions and value formatting.
This method let us concatenate elements within a string through positional
↳ formatting.
string.format(arguments)
"""

####printing values in between a string####
Name1 = "palash"
Age1 = 35
print('My name is: {}, and my age is {}'.format(Name1, Age1)) # Position-based
↳ format function
print('My name is: {0}, and my age is {1}'.format(Name1, Age1)) # index-based
↳ format function
print('My name is: {b}, and my age is {a}'.format(a=Name1, b=Age1)) #
↳ reference-based format function
```

```
[ ]: """
Formatting with % Operator is also possible.
"""

x = 73 ## integer use %d
variable_a = 'H' ## use %c to print charecter
y = 33.98 ## for float Use %f
print("One is integer: %c and other float %f, and another charecter is
↳ %c"%(x,y,variable_a))
"""

--- American Standard Code for Information Interchange.
--- ASCII was the first major character encoding standard for data processing.
--- In standard ASCII-encoded data, there are unique values for 128 alphabetic,
↳ numeric or special additional
characters and control codes.
```

| Dec | Char                   | Dec | Char  | Dec | Char | Dec | Char |
|-----|------------------------|-----|-------|-----|------|-----|------|
| 0   | NUL (null)             | 32  | SPACE | 64  | @    | 96  | `    |
| 1   | SOH (start of heading) | 33  | !     | 65  | A    | 97  | a    |

|                                |       |      |         |
|--------------------------------|-------|------|---------|
| 2 STX (start of text)          | 34 "  | 66 B | 98 b    |
| 3 ETX (end of text)            | 35 #  | 67 C | 99 c    |
| 4 EOT (end of transmission)    | 36 \$ | 68 D | 100 d   |
| 5 ENQ (enquiry)                | 37 %  | 69 E | 101 e   |
| 6 ACK (acknowledge)            | 38 &  | 70 F | 102 f   |
| 7 BEL (bell)                   | 39 '  | 71 G | 103 g   |
| 8 BS (backspace)               | 40 (  | 72 H | 104 h   |
| 9 TAB (horizontal tab)         | 41 )  | 73 I | 105 i   |
| 10 LF (NL line feed, new line) | 42 *  | 74 J | 106 j   |
| 11 VT (vertical tab)           | 43 +  | 75 K | 107 k   |
| 12 FF (NP form feed, new page) | 44 ,  | 76 L | 108 l   |
| 13 CR (carriage return)        | 45 -  | 77 M | 109 m   |
| 14 SO (shift out)              | 46 .  | 78 N | 110 n   |
| 15 SI (shift in)               | 47 /  | 79 O | 111 o   |
| 16 DLE (data link escape)      | 48 0  | 80 P | 112 p   |
| 17 DC1 (device control 1)      | 49 1  | 81 Q | 113 q   |
| 18 DC2 (device control 2)      | 50 2  | 82 R | 114 r   |
| 19 DC3 (device control 3)      | 51 3  | 83 S | 115 s   |
| 20 DC4 (device control 4)      | 52 4  | 84 T | 116 t   |
| 21 NAK (negative acknowledge)  | 53 5  | 85 U | 117 u   |
| 22 SYN (synchronous idle)      | 54 6  | 86 V | 118 v   |
| 23 ETB (end of trans. block)   | 55 7  | 87 W | 119 w   |
| 24 CAN (cancel)                | 56 8  | 88 X | 120 x   |
| 25 EM (end of medium)          | 57 9  | 89 Y | 121 y   |
| 26 SUB (substitute)            | 58 :  | 90 Z | 122 z   |
| 27 ESC (escape)                | 59 ;  | 91 [ | 123 {   |
| 28 FS (file separator)         | 60 <  | 92 \ | 124     |
| 29 GS (group separator)        | 61 =  | 93 ] | 125 }   |
| 30 RS (record separator)       | 62 >  | 94 ^ | 126 ~   |
| 31 US (unit separator)         | 63 ?  | 95 _ | 127 DEL |

"""

ascii\_code = 'A'

v = ord(ascii\_code) ## pass the parameter and get its corresponding ascii value

print(ord(ascii\_code)) # ord () can be used to get the corresponding ASCII

↳value

```
[ ]: name = "Palash"
```

```
name[5]
```

```
print(len(name)) # len() is system function that return the number of
```

↳character of a string

```
x = 10 * name
```

```
print(x)
```

```
[ ]: """Write a python program to print your name sum of ascii char/length times.
```

↳Make sure to print your name

in a new line each time.

Create two files: assign1.txt {your program}  
                    readme1.txt (explanation of your code)  
                    keep this two files in a folder. name the folder with your\_  
↪name\_roll. make a zip.  
                    and upload. """

```
[ ]: # name = "Palash"
# length = len(name)
# sum = ord(name[0]) + ord(name[1]) + ord(name[2]) + ord(name[3]) + ord(name[4]) +
↪ord(name[5])
# print("The sum of ascii", sum)
# d = (sum//length)
# print("D:", d)
# name = name + '\n' # concatenation operator
# y = d * name
# print(y)
#1st time: Palash + '\n'
#2nd time: Palash + '\n'

ascii_value = 65 # Change this to the ASCII value you want to convert
character = chr(ascii_value)

print("Character:", character)
```

```
[ ]: lowercase_char = input("Enter a lowercase character: ") #Assuming one charecter
↪will be given as input
uppercase_char = chr (ord(lowercase_char) - 32)
uppercase_char
```

```
[ ]: ##### List #####
#1. list are used to store multiple values in a single variable.
#2. List is a kind of data type, collection.
#3. List items are ordered (maintain sequence, indexing is available).
#4. List is mutable (changeable).
#5. It also allows duplicates.
#6. For creating a list, we will use square braces.
#7. Each elements of a list is called an item.

nums = [] # empty list
num1 = [25,12,36,95,14, 25, 25] # created a list of homogeneous values (int)
#print(num1[-6])
#print(num1[1:5]) # accessing a portion of a list either by positive or by
↪negative indexing
#print(num1[-5:])
```

```

name = ["Palash", "IITJ", "Sachin"] # list of strings
# #print(name[1])

values = ["Palash", "35", "100000.667"] # we want to create a list of
    ↳ heterogeneous data
values[0] = "John" # a list is mutable or changable.
values

mis = [nums, num1, name] # list of list or nested list
#mis[2][2] # indexing is also possible in nested lists.
# #####some function on a list #####
# num1.append(100) # append function is used to insert at the last of the list
# num1.insert(1, 200) # using insert you can insert an element to desired
    ↳ location.
# num1.remove(25) # remove function removes an element by its value and removes
    ↳ the first occurrence.
# num1.remove(25)
# num1.pop(1) # pop is used to remove an element by the index value
# num1.pop() # Pop with no index deletes from the last.
# print(num1)
# del num1 [2:] # delete is used to delete a portion of a list [exact position:
    ↳ delete position+1]
# print(num1)
# min(num1) # return the minimum
# max(num1)
# sum(num1)
# num1.sort()
values.sort() # ascending order
values
num1.sort(reverse=True) # Descending order
len(name) # length of a list
num1.clear() # clear the entire list
num1.extend([4,5,6])
num1

```

```

[ ]: """
Many logics could be there
"""

# merge two list
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
merged_list = list1 + list2
print("Merged list:", merged_list)

```

```

[ ]: ##### Tuple #####
#1. single variable can store multiple data.
#2. It is also ordered.

```

```

#3. It is not mutable.
#4. Normally faster than a list
t = (23, 20, 6, 5, 20) # tuple is created using the round braces (during
↳ creation, we should be careful about the braces).
print(t[2]) # Indexing is supported so it is ordered. For accessing, always use
↳ square braces.
print(type(t)) # type is a library function so round braces.
#t[2] = 10 # tuples are not mutable or changable.
t1 = ("Palash", "35")
print(t.count(20)) # count is used to count the number occurrences of a value.
print(t.index(6)) # index returns the first occurrence of a value
t3 = (t, t1) # tuple of tuples is also possible
print(t3)

```

```

[ ]: ##### Set #####
#1. A set is collection of unique elements.
#2. for creating a set we will use a curly braces.
#3. Set is not ordered.
s = {2, 3, 1, 8, 5, 96, 2} # duplicates are not allowed in a set
s1 = {2, 4, 5, "palash"}
type(s)
print(s1.intersection(s))
s1.add(3)
s1.add(3)
print(s) #Indexing is not allowed because it does not have any order.
s1.

```

```

[ ]: ##### Dictionary #####
#1. creating dict we will use curly braces.
#2. Dict stores the key-value pairs.
#3. the keys are immutable and unique.

data = {1: "palash", 2: "john", 3: "hemangee"}
print(data[3]) # accessing data is done based on the key value
print(data.get(4, "Data not present"))
data [1] = "ABC"
print(data)
### You can add data like this #####
data [4] = "Rahul"
print(data)
### Delete data####
del data[4]
print(data)
##### creating a dict from two lists #####
keys = ["Palash", "John", "Hemangee"]
values = ["CS", "Networks", "Maths"]
data1 = dict(zip(keys, values)) # zip function and typecasting is used

```

```

print(data1)
##### creating a dict from two tuples #####
keys = ("Palash", "John", "Hemangee")
values = ("CS1", "Networks1", "Maths1")
data2 = dict(zip(keys, values)) # zip function and typecasting is used
print(data2)
##### creating a dict from two sets #####
keys = {"Palash1", "John1", "Hemangee1"}
values = {"CS1", "Networks1", "Maths1"}
data3 = dict(zip(keys, values)) # zip function and typecasting is used
print(data3)
##### Complex and nested dictionary #####
city = {"High Court": 93256613621, "shopping mall": ['PVR', "City Center"],
        ↪ "restaurants": {"chinese": "yochina", "Indian": ["BBQ", "Kasturi"]}}
print (city["restaurants"]['Indian'][0])

```

```

[ ]: """
Range function:
--- The range() function is used to generate a sequence of numbers.
--- It accepts integers and returnd a range object.
--- Syntax: range(start, stop, step)
--- start: Optional, from where to start, Default is 0.
--- stop: Required, where to stop
--- step: Optional, an integer specifying the increament, Default is 1.
"""

v_r = range(20) #The range is a built-in function that return a range object
↪ that consists of series of integers.
print(type(v_r)) # range type object
print (v_r)
print (list(v_r)) # trying to convert a range object to list (type casting).
↪ for type casting we have used list() function.
l1 = list(v_r)
print(l1)
l1 = list(range(5,20,2)) # will go up to max-1
print(l1)
v_r = tuple(range(5,20,5)) # set for set
print (v_r)
# print(type(v_r))

```

```

[ ]: """
--- It is used for condition checking.
--- Indentation is mandatory in python to define the blocks of statements.
--- In python, proper indentation needs to be maintatined. One tab is four
↪ spaces.
--- By default, you may get the indentations in IDEs.
--- Round braces in the condition are optional.
--- elif will check the condition if the previous conditions are false.

```

```

--- else is used for the remaining cases.
Syntax:
if <condition>:
    statement

"""
i = 10
if (i > 15):
    print("10 is less than 15")
print("I am Not in if")
##### Conditional execution using If-else #####
# a = 5
# b = 41
# if a<b: # Boolean values can be used as conditiona as well# in case of
    ↳python : is mandatory. If you keep any non-zero value that is evaluated as
    ↳true.
#     print("We are in if block") # intendentation are used to create block or
    ↳suite
#     print("b is greater than a")
# else : # First make it if and say the differences
#     print("We are in else block")
#     print("a is greater than b")
# print("Out")

```

```

[ ]: x = 88
y = 8
if x < y:
    print("x is lesser than y")
if x!=y:
    print("x and y is not equal")
if x>y:
    print("x is greater than y")
else:
    print ("x and y is equal")
print("done!!!")

```

```

[ ]: x = 88
y = 8
if x < y:
    print("x is lesser than y")
elif x!=y:
    print("x and y is not equal")
elif x>y:
    print("x is greater than y")
else:
    print ("x and y is equal")
print("done!!!")

```

```
[ ]: ### Check if a number is even or odd:
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("The number is even")
else:
    print("The number is odd")
```

```
[ ]: #Find the largest of three numbers:
a = int(input("Enter the first number: "))
b = int(input("Enter the second number: "))
c = int(input("Enter the third number: "))
l1 = []
l1.append(a)
l1.append(b)
l1.append(c)
if a > b and a > c:
    print("The largest number is", a)
elif b > a and b > c:
    print("The largest number is", b)
else:
    print("The largest number is", c)
print(max(l1))
```

```
[ ]: #Check if a year is a leap year:
year = int(input("Enter a year: "))

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print("It's a leap year")
else:
    print("It's not a leap year")
```

```
[ ]: #Determine the grade based on a score:
score = int(input("Enter the score: "))

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```

```
[ ]: ## Simple calculator
operation = input("Enter the operation (+, -, *, /): ")
```

```

num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

if operation == '+':
    result = num1 + num2
elif operation == '-':
    result = num1 - num2
elif operation == '*':
    result = num1 * num2
elif operation == '/':
    if num2 == 0:
        print("Division by zero is not allowed")
    else:
        result = num1 / num2
else:
    print("Invalid operation")

print("Result:", result)

```

```

[ ]: ##### nested if-else#####
name = "IITJ"
name1 = "AIIMs"
deptcode = 5 # assignment, meaning that we are storing the value in the variable
if name == "IITJ":
    if(deptcode == 1): # == is a conditional operator, we check whether the
        ↪value of deptcode is 1
        print("CSE")
    elif(deptcode == 2):
        print("AID")
    else:
        print("NON Computer")
if name1 == "AIIMs":
    if(deptcode == 1):
        print("radiology")
    elif(deptcode == 2):
        print("cardiology")
    else:
        print("General")
print("thanks!!!")

```

```

[ ]: ##### short hand if #####
# when you have only one statement you can use short hand if
# a = 50
# if a>10: print("Hey")
b=3
if b==3:
    print("hello")

```

```

else :
    print ("hi")

# b = 3
# print("hello") if b==2 else print ("hi") ## body of if is in the left and the
↳the right.
# a=4
# b=3
# if b==2:
#     print ("hello")
# else:
#     if (a==b):
#         print ("hi")
#     else:
#         print ("welcome")

# print("hello") if b==2 else print ("hi") if (a==b) else print ("welcome")

```

```

[ ]: ##### While loop #####
"""
With while loop we can execute a set of statements as long as a condition is
↳true.
while is a keyword
General Syntax

while condition: (the loop ends on the terminating condition)
    statement 1
    statement 2
"""

# i = 0 # intialization
# while i<10: # it will execute for i=0--->1.....9, after that i --->10
↳(terminating condition)
#     print("i =", i)
#     i+=1 # increament or a decreament i = i+1
# print("done")

# i = 10 # intialization
# while i>0: # it will execute for i=10,9,8,7.....1, after that i --->0
↳(terminating condition)
#     print("i =", i)
#     i-=1 # increament or a decreament i = i-1
# print("done")

# import time # we have imported this module to get access of the sleep()

```

```

# i = 0 # initialization
# while i<10: # so the condition will always be true ---> infinite loop
#     print(". ", end = "") # if you want your next print to be printed in same
    ↳line write end = ""
#     #time.sleep(1) # sleep function ---> sending this process to running
    ↳state to waiting
#     #i = i+1

# print("done")

# i = 0
# while i<10:
#     i+=1
#     if i==5:
#         break
#     print(i)

# i = 0
# while i<10:
#     i+=1
#     if i in [2,4]:
#         continue
#     print(i)
##### in while loop we can use else statement #####

i = 1
while i<10:
    print(i)
    i+=1
else:
    print("Unlike other languages, while can have else")

# i = 1
# j = 1
# while i<=5:
#     print("IITJ ", end = "")
#     while j<=4:
#         print("CSE ", end = "")
#         j += 1
#     i+=1
#     j=1
#     print("\n")

```

```

[ ]: # Calculate the sum of numbers from 1 to 10 using a while loop
total = 0
current_number = 1
while current_number <= 10:

```

```
total += current_number
current_number += 1
print("The sum of numbers from 1 to 10 is:", total)
```

```
[ ]: # An example of an infinite loop
x = 1
while x :
    print("This is an infinite loop!")
```

```
[ ]: # Ask the user to enter a positive number
number = int(input("Enter a positive number: "))
while number <= 0:
    print("That's not a positive number.")
    number = int(input("Enter a positive number: "))
print("You entered:", number)
```

```
[ ]: # Menu-driven program using a while loop
while True:
    print("Menu:")
    print("1. Option 1")
    print("2. Option 2")
    print("3. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        print("You selected Option 1")
    elif choice == 2:
        print("You selected Option 2")
    elif choice == 3:
        print("Exiting the program")
        break
    else:
        print("Invalid choice. Please enter a valid option.")
```

```
[ ]: # Calculate the factorial of a number using a while loop
number = int(input("Enter a number to find its factorial: "))
fact = 1
count = 1

while count <= number:
    fact *= count
    count += 1

print(f"The factorial of {number} is: {fact}")
```

```
[ ]: # Print a pattern using a while loop
rows = 5
```

```

current_row = 1

while current_row <= rows:
    print('*' * current_row)
    current_row += 1

```

```

[ ]: # Print a pattern using a while loop
rows = 5
current_row = 1
j = 1
k = 1
while current_row <= rows:
    while j<=current_row:
        print(k, end = " ")
        j+=1
        k+=1
    j = 1
    current_row += 1
    print()

```

```

[ ]: # Print a pattern using a while loop
rows = 5
current_row = 1
j = 1
count = 1
while current_row <= rows:
    while j<=current_row:
        print(count,end = " ")
        j+=1
        count+=1
    j = 1
    current_row += 1
    print()

```

```

[ ]: ##### Loops #####
"""
A loop is a sequence of instructions that is continuously repeated untill a
↳certain condition is reached.
Primary two loops:
1. for loop
2. while loop
"""
v_r = range(10) #The range is a built-in function that return a range object
↳that consists of series of integers.
# print(type(v_r)) # range type object
# print (v_r)

```

```
# print (list(v_r)) # trying to convert a range object to list (type casting).
↳for type casting we have used list() function.
##### for loop #####

for a in v_r: # for is a reserved keyword. in is a operator to checks if a
↳value is in a collection of values.
    print(a)
print ("Hi")
```

```
[ ]: v_r1 = range(5,11)
# for a in v_r1: # for is a reserved keyword. in is a operator
#     print(a)
#     print ("Hi") # for each element of the sequence the loop will iterate,
↳loop will iterate means whatever you will write inside a loop that will
↳execute.

# v_r2 = range(11)
# for a in v_r2:
#     print(a)

# v_r2 = range(11,0,-2)
# for a in v_r2:
#     print(a)

# Name = 1234 # for loop also iterate over string object
# for a in Name:
#     print("Hi")
# num = "456" # for loop also iterate over string object
# for a in num: # for loop does not iterate over numbers
#     print(a)
#     print("Hi")
# l1 = [2,5,1,7,89] # iterable objects: range object, string, collections # try
↳on set and tuple
# for a in range(5): # iterating on range object
#     print(l1[a])
# for a in l1: # iterating on directly on the list object
#     print(a)
# data = {1: "palash", 2: "john", 3: "hemangee"}

# for v in data.keys(): # we can iterate on keys, values or by default is keys
#     print(v)
#     print(data[v])

# Nested for loop example
# for i in range(3):
#     for j in range(3):
#         print(f"{{i}}, {{j}}")
```

```
[ ]: ##### break, continue pass#####
# fruits = ["apple", "orange", "banana", "cherry", "mango"]

# for x in fruits: # iterable object is list, so for every element of the list
    ↳ loop iterates
#     if (x == "banana"): # == is a conditional operator, it does not store a
    ↳ value
#         break # break is used to stop/terminate the immediate loop
#         #continue # continue will skip the present iteration
#     print("I love "+x)

# print ("Done "+ x)

# l2 = [2,5,6,10,11]
# for i in l2:
#     if i in [5,6]:
#         continue
#     print(i)

# for i in range(50):
#     if (i%2 == 0):
#         pass #signifying a blank body
#     else:
#         print(i)
```

```
[ ]: # Example: Calculating the sum of numbers in a list using a for loop

# Define a list of numbers
numbers = [1, 2, 3, 4, 5]

# Initialize a variable to store the sum
sum_of_numbers = 0

# Use a for loop to iterate over the numbers in the list
for num in numbers:
    # Add the current number to the sum
    sum_of_numbers += num

# Print the result
print(f"The sum of numbers is: {sum_of_numbers}")
```

```
[ ]: # Example: Creating a multiplication table using a for loop

# Specify the number for the multiplication table
number = 7
```

```
# Use a for loop to generate the multiplication table for the specified number
print(f"Multiplication table for {number}:")

for i in range(1, 11):
    result = number * i
    print(f"{number} x {i} = {result}")
```

```
[ ]: ##### Using a temporary variable #####
# values = ["palash", 35, 56.8]
# temp = values[2] ### 56.8
# values[2] = values[0] ## palash
# values[0] = temp ### 56.8
# print(values)
##### without using a temporary variable #####
# values = ["palash", 35, 56.8]
# print(values)
# values.append(values[2]) #["palash", 35, 56.8, 56.8]
# values[2] = values[0] #["palash", 35, "palash", 56.8]
# values[0] = values[3] #[56.8, 35, "palash", 56.8]
# values.pop() #[56.8, 35, "palash"]
# print(values)
```

```
[ ]: s={22,25,14,21,7,100,300}
print(s) # Doesst follow sequence.
s1 = {22,2,14,21,7,258}
print(s1.intersection(s))
c = len(list(s1.intersection(s)))
if (c%2==0):
    print("Even")
else :
    print("Odd")
```

```
[ ]: ##### nested loops #####

i = 0
j = 0
while i<5:          # i =0--->1--->2--->3--->4
    ↪ j=0--->1--->2--->3--->4//// j=0
    print("IITJ")
    while j<5:
        print ("rocks")
        j+=1
    i+=1
    j=0 ##### resetting the condition
print("Exit!!!!")
```

```
[ ]: for i in range(5):    # i= 0,1,.....,4
      print("IITJ")
      for j in range (5): # for each value of i, this loop will execute 5 times
          print("rocks")
```

```
[ ]: x = [1, 2]
     y = [4, 5]

     for i in x: # for each value of i both the values of j will be taken
         for j in y:
             print(i,j)
```

```
[ ]: # Example program using nested for loops for a pattern

     # Define the number of rows and columns
     rows = 5
     columns = 5

     # Use nested for loops to print a square pattern
     for i in range(rows):
         for j in range(columns):
             print("* ", end="")
             print() # Move to the next line after each row
```

```
[ ]: ##### Function #####
     """
     1. In Python, a function is a group of statements that performs some task. It
        ↳ returns some value at
        end.
     2. Procedure are similar to functions but it should not return a value. It
        ↳ only performs the
        tasks. But the control will be returned to the calling function.
     3. In both, we can have parameters.
     4. A parameter is the variables listed inside the parenthesis in the function
        ↳ definition.
     5. An argument is the value that is sent to the function when it is called.
     6. Function increases the code reusability and reduce the LOC.
     """

     def sum_of_two_numbers():#we are creating a new function.def is used to create
        ↳ function definition
        a = 3
        b = 5
        c = a+b
        print ("the sum is ", c)

     def sum_of_two_numbers1(a, b): # parameters are catching the values based on
        ↳ the positions
```

```

    print ("the sum is ", a+b) # inside the function, we are printing the values

def sum_of_two_numbers2(a, b): # we are throwing the result
    return a*b

sum_of_two_numbers() # we are calling the function
sum_of_two_numbers()
sum_of_two_numbers()

sum_of_two_numbers1(2,3) # while calling the function, we pass arguments
sum_of_two_numbers1(3,4) # throwing the value
sum_of_two_numbers1(4,5)

print("the mul is ",sum_of_two_numbers2(12,13))
result = sum_of_two_numbers2(13,14) # results = 27
print ("the mul is ", result)
#sum_of_two_numbers2(14,15)

```

```

[ ]: def add_sub (x, y):
    c = x + y
    d = x - y
    return c, d
r1, r2 = add_sub(5, 4)
print(r1, r2)

```

```

[ ]: def update(x):
    print(id(x))
    x = 8
    print(id(x))
    print("x ", x)
a = 10
print(id(a))
update(a)
print("a", a)

```

```

[ ]: def update(lst):
    print(id(lst))
    lst[1] = 25
    print(id(lst))
    print("lst ", lst)

lst1 = [10, 20, 30]
print(id(lst1))
update(lst1)
print("lst1 ", lst1)

```

```
[ ]: def my_details (name, age, loc = "Delhi"): #user-defined function. You can also
    ↪ create default value
    print("My name is: ", name)
    print("My age is: ",age)
    print("My location is: ", loc)

my_details (age = 35, name = "palash", loc = "kolkata") # here the arguments
    ↪ are keyword-based
```

```
[ ]: ##### Another Example (passing list parameter) #####

def check_even (full_list): # function definition
    even_found = [] # empty list
    for nums in full_list: # every elemnts will checked
        if nums % 2 == 0: # checking whether it is divisible by 2
            even_found.append(nums) # if divisible add it to the empty list
        else:
            pass # do nothing
    return even_found

limit = int (input("Please enter the upper limit for natural numbers: "))
    ↪ #typecasting
# input function is used to take user input and it return string object.

natural_number = list(range(1,limit+1)) # typecasting range object----> list
    ↪ object
print(natural_number)
print(check_even(natural_number))
print(check_even([20,30,100,3,5,8]))
```

```
[ ]: ##### Lambda Function #####
"""
1. Python Lambda functions are anonymous functions which means that the
    ↪ functions are without name
2. Lambda keyword is used to create such function.
Syntax:
lambda arguments: expression
5. Multiple arguments can be there.
6. Single line.
"""
#a = 10
# l_func = lambda a: a+10
# l_func1 = lambda b: b*9
# print (type(l_func))
# print(type(a))
# print (l_func(100))
# print (l_func1(100))
```

```
aa1 = lambda a,b: (a+b, a*b)
print("the answer is:", aa1(10,30))
```

```
[ ]: def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    if y != 0:
        return x / y
    else:
        return "Error: Cannot divide by zero."

# Get input from the user
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
operation = input("Choose operation (+, -, *, /): ")

# Perform the selected operation
if operation == '+':
    result = add(num1, num2)
elif operation == '-':
    result = subtract(num1, num2)
elif operation == '*':
    result = multiply(num1, num2)
elif operation == '/':
    result = divide(num1, num2)
else:
    result = "Invalid operation."

# Print the result
print(f"The result of {num1} {operation} {num2} is: {result}")

[ ]: def calculate_rectangle_area(length, width):
    area = length * width
    return area

# Get input from the user
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))
```

```

# Call the function to calculate the area
area = calculate_rectangle_area(length, width)

# Print the result
print(f"The area of the rectangle with length {length} and width {width} is:␣
↪{area}")

```

```

[ ]: def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Get input from the user
number = int(input("Enter a number to calculate its factorial: "))

# Call the factorial function and print the result
result = factorial(number)
print(f"The factorial of {number} is: {result}")

```

```

[ ]: def is_prime(number):
    if number <= 1:
        return False
    for i in range(2, int(number**0.5) + 1): # 2 to the square root of the␣
↪number and checks
                                                #if any of these values divides␣
↪the number evenly
        if number % i == 0:
            return False
    return True

# Get input from the user
num = int(input("Enter a number to check if it's prime: "))

# Call the function to check if the number is prime
prime_check = is_prime(num)

# Print the result
if prime_check:
    print(f"{num} is a prime number.")
else:
    print(f"{num} is not a prime number.")

```

```

[ ]: ##### variable number of arguments (dynamically) #####

# def my_mul_add(*args): # function defination of a user defined function
#     i = 0

```

```

#     mul = 1
#     while (i<len(args)): ##### i=0, len(args)---->4
#         mul = mul * args[i] ### i=0--> mul = 1 * 10, i=1--> mul = 10 * 12, i=2--> mul = 120 * 3
#                                     ### i=3--> mul = 360 * 4
#         i+=1
#     print ("The mul result is: ", mul)
#     return sum(args) # I have used a library func (sum)

# print("The sum is: ",my_mul_add(10,12,3,4)) # I have called the function

# def my_key_value(**kvp):
#     #print("The collection is: ", kvp)
#     for a in kvp:
#         print(kvp[a], end=" ")

# my_key_value(name = "Palash", age=35, loc = "Kolkata")
# you can pass any number of key-value pairs.

```

```

[ ]: ##### Nested Functions #####

def i_am_outsider():
    print ("I am the outsider function")
    def i_am_insider(): # insider function
        print("I am the inner function")

        i_am_insider() # call is ok. Why? Because we are calling from inside the
        ↪function
        i_am_insider()

i_am_outsider()
i_am_insider() # out of scope can not be called

```

```

[ ]: # def is_even(n):
#     if (n%2==0):
#         a = n
#     else:
#         a = False
#     return a

# nums = [2,3,4,5,6,7,8]
# evens = list (filter(is_even, nums)) # filter-->library function

# print(evens)

```

```

nums = [2,3,4,5,6,7,8]
evens = list (filter(lambda n: n%2==0, nums)) # filter-->library function

print(evens)

```

```

[ ]: def smallest_palindromic_string(s):
    def is_palindrome(string):
        return string == string[::-1]

    for i in range(len(s)):
        if is_palindrome(s[i:]):
            prefix = s[:i]
            suffix = s[i:]
            additional_chars = prefix[::-1]
            return s + additional_chars

    return s + s[:1][::-1]

# Example usage
input_string = "abcd"
result = smallest_palindromic_string(input_string)
print(result)

```

```

[ ]: ##### Object oriented Programming (OOPs) #####
    """
    1. Class has properties + behaviour.
    Properties-----> variables or attributes.
    Behaviour -----> Methods/functions.

    What is a class?
    A class can be considered as a template definition of the methods and variables_
    ↪ in a particular
    kind of object.
    It is a blueprint for an object.

    What is object?
    Objects are the instances of a particular class.

    The roles of a constructor (it is a special function) is

        1. creating a object.
        2. intializing the attributes.
    Constructors are called by the class name.
    """
    class computer:
        def config(self):
            print("i5 16GB 1 TB");

```

```

c1 = computer();
# config() # you can not directly call this. Why?
c1.config(); # behind the scene-----> computer.config(c1)
computer.config(c1);
print(type(c1));
name = "Palash"
print(type(name));

# class student: # keyword class. Class always has a special function---->
↳ Constructor

#     def __init__(self, name, roll, marks): # Attributes are the properties.
↳ inint is a special method.
#         self.name = name
#         self.roll = roll
#         self.marks = marks

#     def print_data(self):
#         print("The student's name is :", self.name)
#         print("The student's roll is :", self.roll)
#         print("The student has obtained:", self.marks)
#         print("-----")

# stud1 = student ("palash", 1, 85) # Calling Constructor by the class name
# stud2 = student ("John", 2, 95) # object creation
# #stud3 = student ("aa", 3, 75)
# stud1.print_data()
# stud2.print_data()
# print_data()

```

```

[ ]: class student:

    def __init__(self):
        self.name ="palash"
        self.roll = 1
    def show(self1):
        a=10
        print("The name is: ", self1.name)
        print("the Roll is:", self1.roll)
        print(a)

a=20
s1 = student() # we are making the object and default constructor is getting
↳ called
s2 = student()
print(id(s1))

```

```

print(id(s2))
s1.show()
s2.show()
print(a)
s2.name = "John"
s2.show()
student.show(s1)

```

```

[ ]: ##### Comparing two objects #####
class student:

    def __init__(self):
        self.name = "palash"
        self.roll = 3

    def update(self):
        self.roll = 36

    def compare(self, obj1):
        if self.roll == obj1.roll:
            return True
        else:
            return False

s1 = student()
s2 = student()
#s1.update()
if s2.compare(s1):
    print("They are same")
else:
    print("they are not same")

```

```

[ ]: class ComparableObject:
    def __init__(self, value):
        self.value = value

    def __lt__(self, other):
        return self.value < other.value

obj1 = ComparableObject(10)
obj2 = ComparableObject(20)

if obj1 < obj2:
    print("obj1 is less than obj2")
else:
    print("obj1 is greater than or equal to obj2")

```

```
[ ]: ##### Types of variables #####
"""
1. class variable: They are shared across all objects.
2. Instance variable: These are the variables unique to each instance/object.

"""

class bike:
    wheels = 2 #class variable

    def __init__(self):
        self.cc = 350
        self.topspeed = 120

    def show_details(self):
        print("My cc is :", self.cc)
        print("My topspeed is :", self.topspeed)
        print("The number of wheels: ", self.wheels)
        print("-----")

b1 = bike()
b2 = bike()
b1.show_details()
b2.show_details()
b2.cc = 150

bike.wheels = 3
b1.wheels =4

b1.show_details()
b2.show_details()
```

```
[ ]: ##### Type of methods #####
"""
1. Instance methods: Deals with the instance variables of the objects.
2. Class methods: Deals with the class variables and can be called on class_
   ↳ itself
3. Static methods: Normally do not deals with the class variables/instance_
   ↳ variable. Generic/isolated
   operations can be implemented.
"""

class Student:

    college = 'IITJ'

    def __init__(self, m1,m2,m3):
        self.m1 = m1
```

```

        self.m2 = m2
        self.m3 = m3

    def avg(self):      # instance method as we are passing self-->means it
    ↪ belongs to some object
        return (self.m1+self.m2+self.m3)/3
    @classmethod        #decorators
    def get_coll_name(cls): # Class methods --> deals with the class variables,
    ↪no self (as we are not dealing with obj)
        print("The college is :", cls.college)    # cls refers to class here
    @staticmethod      #decorators
    def info():         # no self, no cls-----> make it blank for the static
    ↪methods
        print("This is a student class.....")

s1 = Student(23,24,25) # constructor
s2 = Student(35,36,37)

print(s1.avg())
print(s2.avg())

#print(Student.avg()) # error, because it does not know which object to print.
    ↪you can pass object as parameter.

Student.get_coll_name() # error, you need a decorator.
s1.get_coll_name()

Student.info()
s1.info()
#Student.info(s2) # Error as passing objects
#info() #error

```

```

[ ]: class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

    def can_vote(self):
        return self.age >= 18

# Example usage:
# Create objects of the Person class
person1 = Person("Alice", 25)

```

```

person2 = Person("Bob", 17)

# Call the introduce method for each person
person1.introduce()
person2.introduce()

# Check if each person can vote
print(f"{person1.name} can vote: {person1.can_vote()}")
print(f"{person2.name} can vote: {person2.can_vote()}")

```

```

[ ]: class dog:

    def __init__(self):
        self.a = 20

    def show_data(self):
        print(self.a)

```

```

class cat:

    def __init__(self):
        self.b = 30

    def show_data1(self):
        print(self.b)
        d1 = dog()
        print(d1.a)

# d1 = dog()
c1 = cat()

# d1.show_data()
c1.show_data1()

```

```

[ ]: ##### inner class #####

class Student1:

    def __init__(self, name1, roll1):
        self.name = name1
        self.roll = roll1
        self.lap = self.Laptop("dell", "i5", 16)

    def show(self):
        print(self.name, self.roll, self.lap.brand, self.lap.cpu, self.lap.RAM)

    class Laptop:

```

```

    def __init__(self, brand, cpu, RAM):
        self.brand = brand
        self.cpu = cpu
        self.RAM = RAM

s1 = Student1("Palash", 1)
s2 = Student1("John", 2)
s1.show()
s2.show()

lap1 = Student1.Laptop("dell", "i5", 16) #u can also create an object of inner class outside the outer class but u need outer class name to call
print(lap1.RAM)
print(s1.lap.RAM)
lap1.RAM = 17
print(lap1.RAM)

```

```

[ ]: ##### Inheritance #####
"""
1. Single-level inheritance. (Parent--->child, Super---> sub)
2. Multi-level inheritance. (Grand Parent---> Parent----> child)
3. Multiple inheritance. (Multiple parent class)

"""
##### Single level inheritance #####

class A:

    def property1(self):
        print ("I am property 1 from class A")

    def property2(self):
        print ("I am property 2 from class A")

class B (A): #B inherits A

    def property3(self):
        print ("I am property 3 from class B")

    def property4(self):
        print ("I am property 4 from class B")

b = B()
b.property1()
b.property2()
b.property3()

```

```

b.property4()
print("-----")
a = A()
a.property1()
a.property2()
a.property3()      # error child's property is not there in parent.
a.property4()      # error child's property is not there in parent.

```

```

[ ]: ##### Constructors in inheritance #####
class A:

    def __init__(self):
        print("In A's Constructor")

    def property1(self):
        print ("I am property 1 from class A")

    def property2(self):
        print ("I am property 2 from class A")

class B (A):

#     def __init__(self):      # If b's constructor is not present and A's one
#         print("In B's Constructor")
#         present then A's constructor will be called.

    def __init__(self):
        super().__init__()      # Using Super(), you can call super class methods
#         + constructor.
        super().property1()      # other methods can also be called.
        print("In B's Constructor")

    def property3(self):
        print ("I am property 3 from class B")

    def property4(self):
        print ("I am property 4 from class B")

#a1 = A()
b1 = B()

```

```

[ ]: ##### Anonther example of single-level inheritance #####
class trainee:

    def __init__(self, tid):
        self.trainee_id = tid

```

```

def show_details_t(self):
    print("Trainee's id is :", self.trainee_id)

class manager(trainee):
    def __init__(self, mgr_id, tid):
        super().__init__(tid)
        self.mgr_id=mgr_id

    def show_details_M(self):
        print("Trainee's id is :", self.trainee_id)
        print("Manager's id is :", self.mgr_id)

t = trainee (23)
m = manager(20, 1)          # Object construction brings some properties from
                             ↪parent class

m.show_details_M()
m.show_details_t()
t.show_details_t()
t.show_details_M()         # error child's property is not there in parent.

```

[ ]: ##### Multi-level inheritance #####

```

class trainee:

    def __init__(self, tid):
        self.trainee_id = tid

    def show_details_t(self):
        print("Trainee's id is :", self.trainee_id)

class project_lead(trainee):

    def __init__(self, pid, tid):
        super().__init__(tid)
        self.p_id = pid

    def show_details_p(self):
        print("Trainee's id is :", self.trainee_id)
        print("Project_lead's id is :", self.p_id)

class manager(project_lead):

    def __init__(self, mgr_id, pid, tid):
        super().__init__(pid, tid)

```

```

        self.mgr_id = mgr_id

    def show_details_M(self):
        print("Trainee's id is :", self.trainee_id)
        print("Project_lead's id is :", self.p_id)
        print("Manager's id is :", self.mgr_id)

#t = trainee (23)
m = manager(20, 1, 2)          # Object construction brings some properties from
    ↪ parent class
p = project_lead(4,5)
m.show_details_M()
print("-----")
m.show_details_p()
print("-----")
m.show_details_t()
p.show_details_M()             # error child's property is not there in parent.
# t.show_details_M()          # error child's property is not there in parent.

```

[ ]: ##### Multiple inheritance #####

```

class trainee:

    def __init__(self, tid):
        self.trainee_id = tid

    def show_details_t(self):
        print("Trainee's id is :", self.trainee_id)

    def test(self):
        print("Test from trainee", self.trainee_id)

class project_lead:

    def __init__(self, pid):
        self.p_id = pid

    def show_details_p(self):
        print("Project_lead's id is :", self.p_id)

    def test(self):
        print("Test from project_lead", self.p_id)

class manager(trainee, project_lead):

    def __init__(self, mgr_id, pid, tid):

```

```

        super().__init__(tid)
        project_lead.__init__(self, pid)
        #self.p_id = pid
        self.mgr_id = mgr_id
        super().test() # Method resolution order
        project_lead.test(self)

    def show_details_M(self):
        print("Trainee's id is :", self.trainee_id)
        print("Project_lead's id is :", self.p_id)
        print("Manager's id is :", self.mgr_id)

#t = trainee (23)
m = manager(20, 1, 2)          # Object construction brings some properties from
    ↳parent class
p = project_lead(55)
m.show_details_M()
print("-----")
m.show_details_p()
print("-----")
p.show_details_p()
print("-----")
#p.show_details_M()          # error child's property is not there in parent.

```

```

[ ]: ##### Polymorphism #####
"""
1. Polymorphism: Poly ----> many, morph ----> forms.
2. Four ways to implement polymorphism are:

    a. Duck Typing.
    b. Operator Overloading.
    c. Method Overloading.
    d. Method Overriding.
"""

```

```

[3]: ##### Duck Typing #####
"""
1. Duck typing is a concept related to dynamic typing, where the type or the
    ↳class of an object is less important than
the methods it defines.

2. When you use duck typing, you do not check types at all. Instead, you check
    ↳for the presence of a given method or attribute.

Analogy: If a bird walks like a duck, quacks like a duck, swims like a duck, we
    ↳call that bird a duck.

```

```

"""
class Sublime:

    def execute(self):
        print("Highlight text using color code")
        print("Compile")
        print("Run")

class Jupyter:

    def execute(self):
        print("Highlight text using color code")
        print("Spell check")
        print("Provide Cells")
        print("Compile")
        print("Run")

class Computer:

    def program (self, IDE): # Dynamic typing of object, you can replace object
        ↪type provided
        IDE.execute()          # execute method should be there. We only see the
        ↪behaviour
                                #(execute()) and we think its an IDE. May be a dog
        ↪class
                                #can have execute but thats not and IDE. But we
        ↪don't care.

i1 = Sublime() # Object of sublime class
i2 = Jupyter() # Object of Jupyter class

sys1 = Computer()
sys1.program(i2)

```

Highlight text using color code  
 Spell check  
 Provide Cells  
 Compile  
 Run

```

[2]: a = 5
      b = 6
      c = "Palash"
      d = "IITJ"
      print(a+b)
      print(int.__add__(a,b))

```

```
print(c+d)
print(str.__add__(c,d))
```

11

11

PalashIITJ

PalashIITJ

```
[ ]: ##### Operator Overloading #####
"""
1. Operator Overloading means giving extended meaning beyond their predefined_
   ↳ operational meaning.
2. You might have noticed that the same built-in operator or function shows_
   ↳ different behavior for
   objects of different classes, this is called Operator Overloading.
3. The method name is the same, but the type of arguments and number of_
   ↳ arguments change.
"""
class student:

    def __init__(self, m1, m2):
        self.m1 = m1
        self.m2 = m2

    def __add__(self, obj):
        m1 = self.m1 + obj.m1
        m2 = self.m2 + obj.m2
        s3 = student(m1, m2)
        return s3

    def show(self):
        print (self.m1, self.m2)

s1 = student (22, 44)
s2 = student (4, 5)

s3 = s1 + s2    # student.__add__(s1, s2)

# s4 = s1 - s2
# s5 = s1 * s2

s3.show()
# s4.show()
# s5.show()
```

```
[1]: ##### Method Overloading #####
class Student:

    def __init__(self, m1, m2):
        self.m1 = m1
        self.m2 = m2

    def sum (self, a, b):
        return a+b

    def sum (self, a, b, c):
        return a+b+c

#     def sum (self, a = None, b = None, c = None):
#         s = 0

#         if a!=None and b!=None and c!=None:
#             s = a+b+c
#         elif a!=None and b!=None:
#             s = a+b
#         else:
#             s = a

#         return s

s1 = Student (5, 6)
print(s1.sum(2,3,7))
print(s1.sum(2,3))
```

12

```
-----
TypeError                                Traceback (most recent call last)
Cell In [1], line 28
    26 s1 = Student (5, 6)
    27 print(s1.sum(2,3,7))
--> 28 print(s1.sum(2,3))

TypeError: Student.sum() missing 1 required positional argument: 'c'
```

```
[ ]: ##### Method Overriding #####

class A:

    def show (self):
        print("I am show from class A")
```

```

class B(A):
    #pass
    def show (self):
        print("I am show from class B")

o = B()
o.show()

```

```

[ ]: ##### Array in python #####
"""
1. Arrays are collection of homogeneous datatypes.
2. Types: i-->signed int, I --> unsigned int, f--> float, d-->double, L --->
    ↳ unsigned Long, l ---> signed long,
u--->char (unicode).
"""

from array import *                # By default arrays are not supported, you
    ↳ need to import the array module.

vals = array('i', [1,2,3,4,5])
vals1 = array('i', [1,2,3,4,6]) # Heterogenous data not supported
#vals = array('I', [1,2,3,-4,5]) # Signed int, both positive and negative
    ↳ values are supported
# vals1 = array('i', [1,2,3,-4,5])

print(vals.buffer_info()) # Returns a tuple with initial address of the array
    ↳ with the number of elements.
print(vals.typecode)      # Returns the typecode of the array i.e. i, I, B ,
    ↳ etc.
vals.reverse()            # Reverse does not return any value. This only
    ↳ reverses the whole array.
print(vals)
print(c[0])

```

```

[ ]: ##### How to print array elements one by one #####
from array import *
vals = array('i', [11,21,31,-4,5])

# for i in range(len(vals)):      # this works
#     print(vals[i])

# for i in vals:                  # this works
#     print(i)

# i = 0                          # this also works
# while i<len(vals):

```

```
#     print(vals[i])
#     i+=1
```

```
[ ]: #####Coping one array to other #####
from array import *
vals = array('i', [1,2,3,-4,5])

newarr = array(vals.typecode, (a*2 for a in vals)) # copied double of each
↪number
print(newarr)
```

```
[ ]: val3 = vals+vals1                # not getting elementwise sums. One is
↪appended with other. Elementwise sum in numpy.
print(val3)
# #vals = array('I', [1,2,3,-4,5]) # Unsigned int, negative value not supported
c = array('u', ['A','B', 'C'])      # u (unicode) --> charecters
# print(vals)
```

```
[ ]: ##### User input to an array #####

from array import *

arr = array('i', [])
n = int(input("Enter the length of an array "))

for i in range(n):
    x = int(input("Enter a value "))
    arr.append(x)

print(arr)
```

```
[ ]: ##### Searching an element in the array #####
from array import *

arr = array('i', [])
n = int(input("Enter the length of an array "))

for i in range(n):
    x = int(input("Enter a value "))
    arr.append(x)

Key = int(input("Enter the value to be searched "))
k = 0
for item in arr:
    if item==Key:
        print("Item found on ", k)
        break
```

```
k+=1
```

```
print("Item found on ", arr.index(Key))
```

```
[ ]: ! pip3 install numpy
```

```
[ ]: ##### numpy #####
"""
1. Helps in dealing with multi-dimensional array.
2. This does not come with the default packages of python. We need to install
   ↳ it separately.
3. Use "! pip3 install numpy" to install numpy.
"""
from numpy import *

vals = array([1,2,3,-4,5.6]) # typecode is not mandatory.
vals1 = array([1,2,3,-4,5.6], float) # Explicit Conversion
print(vals)
print(vals1)
```

```
[ ]: ##### More on arrays #####
"""
--> Arrays can be made by array(), linspace(), logspace(), arange(), zeros(),
   ↳ ones().....
--> NumPy arrays have a fixed size at creation, unlike Python lists (which can
   ↳ grow dynamically).
"""

from numpy import *
arr = array ([1,2,3,4,5])
# print(arr)
print(arr.dtype) # datatype of the values
# arr1 = array ([1,2,3,4.8,5]) # implicit conversion
# print(arr1) # Every value here will be converted into float
# print(arr1.dtype)
# arr2 = array ([1,2,3,4,5], float) # Explicit conversion
# print(arr2)
# arr3 = array ([1.2,2.3,3.1,4.2,8.3], int) # Explicit conversion
# print(arr3)

# arr = linspace(0,15,25) # Parameters are start, stop, parts. Default step is
   ↳ 50. As it is step values are by default float here.
# #arr[1] = 3
# print(arr)
```

```

arr1 = arange(0,15,2) # Parameters are start, stop, steps. Here, it is step not
    ↳ a part.
#arr1[1] = 3
print(arr1)

# arr1 = zeros(15, int) # Array of size 15. All are initialized to 0s. By
    ↳ default, the values are float.
# #arr1[1] = 3
# print(arr1)

# arr1 = ones(15, int) # Array of size 15. All are initialized to 1s. By
    ↳ default, the values are float.
# #arr1[1] = 3
# print(arr1)

# arr = logspace(1,40,5)#Array will be divided into 5 parts. Start: 101 end:
    ↳ 1040. The start to end will be divided into 5 parts.
# #arr[1] = 3
# print(arr)
# print('%0.2f' %arr[1])

```

```

[ ]: ##### More Operations on array #####
from numpy import *
arr = array([1,2,3,4,5])
arr1 = array([1,2,3,4,5])
arr2 = zeros(5, int)
# Want to add a number with each element of the array #####

# ##### 1st approach #####

# for i in range(len(arr)):
#     arr[i] = arr[i] + 5

# print(arr)

# ##### 2nd approach #####
# arr = arr + 10
# print(arr)

# Want to add 2 arrays #####

##### 1st approach #####

for i in range(len(arr)):
    arr2[i] = arr[i] + arr1[i]

```

```

print(arr2)

# ##### 2nd approach #####
arr3 = arr + arr1 ### Vectorized Operation (Was getting appended without numpy_
    ↳right?)
print(arr3)

```

```

[ ]: ##### Mathematical operations #####
from numpy import *
arr = array([1,2,3,4,5])
arr1 = array([6,7,8,9,10])
print(sin(arr)) ##### Sin values of each element #####
print(cos(arr)) ##### Cos values of each element #####
print(log(arr)) ##### log values of each element #####
print(sqrt(arr)) ##### sqrt values of each element #####
print(sum(arr)) ##### sum of each element #####
print(min(arr)) ##### min of each element #####
print(max(arr)) ##### max of each element #####
arr3 = concatenate([arr, arr1])
print(concatenate([arr, arr1])) ##### Concatenating two arrays_
    ↳#####
print(arr3) ##### Concatenating two arrays #####

```

```

[ ]: ##### Coping one array to other #####
"""
Three types of copies-
    1. Aliasing.
    2. Shallow Copy.
    3. Deep Copy.
"""
from numpy import *
arr = array([1,2,3,4,5])
# arr2 = arr # Only Aliasing
# print(arr2)
# print(id(arr), id(arr2)) #In memory, we have only one instance of each_
    ↳element. Only duplicate reference variables are created.

# arr3 = arr.view() # shallow copy ---> changing in one affects the other.
# arr[1] = 90
# print(arr)
# print(arr3)
# print(id(arr), id(arr3)) # Addresses are different i.e. two copies of each_
    ↳elements are now created.

arr3 = arr.copy() # deep copy ---> changing in one does not affect the other.

```

```

arr[1] = 90
print(arr)
print(arr3)
print(id(arr), id(arr3))

```

```

[ ]: ##### Working with matrix in python #####
from numpy import *

arr1 = array([
    [1,2,3,7,8,9],
    [4,5,6,99,88,77]
])

print (arr1)
print (arr1.dtype) # Data type
print (arr1.ndim)  # Number of dimension
print (arr1.shape) # (rows, cols)
print (arr1.size)  # total number of elements

arr = arr1.flatten() # Convert the 2D array into 1D array
print(arr)

arr3 = arr.reshape(3,4) # 3 rows 4 cols ----> Converting 1D array to 2D array
print (arr3)
print ("-----")
arr4 = arr.reshape(3,2,2) # 3 dim 2 rows 2 cols ----> Converting 1D array to
    ↪ 2D array
print (arr4)

```

```

[ ]: ##### Working with matrices #####

from numpy import *

arr1 = array([
    [1,2,3,7],
    [4,5,6,9]
])

m = matrix(arr1) # Converting a 2D array into a matrix format
m1 = matrix('1 2 3;4 5 6;7 8 9') # Converting a string into a matrix format
    ↪ (even this works)
print(m) # Output is same but with this conversion, you can do plenty of
    ↪ additional operations.
print(m1)
print(diagonal(m1)) # print the diagonal elements of the matrix
print(m1.min()) # Finding Minimum of a matrix
print(m1.max()) # Finding Maximum of a matrix

```

```
[ ]: ##### Working with matrices #####
from numpy import *
m = matrix ('1 2 3;4 5 6;7 8 9')
m1 = matrix ('1 2 3;4 5 6;7 8 9')

m3 = m+m1
m4 = m*m1

print("The result matrix is \n", m3)
print("The result matrix is \n", m4)
```

```
[ ]: from numpy import *
R = int(input("Enter the number of rows:"))
C = int(input("Enter the number of columns:"))

mat = []
mat1 = []
print("Enter the entries rowwise:")

for i in range(R):
    a = []
    for j in range(C):
        a.append(int(input()))
    mat.append(a)
m = matrix(mat)

for i in range(R):
    a = []
    for j in range(C):
        a.append(int(input()))
    mat1.append(a)
m2 = matrix(mat1)

for i in range(R):
    for j in range(C):
        mat[i][j] = mat[i][j] + mat1[i][j]
m = m + m2
print("The sum is = \n")
for i in range(R):
    for j in range(C):
        print(mat[i][j], end = " ")
    print()

print("-----")
print("The sum is = \n")
print(m)
```

```
[ ]: ##### Iterator #####
"""
1. Iterator is an object that contains the countable number of values. Iterator
   ↳ in Python is simply an object that can be
   iterated upon. An object which will return data, one element at a time.
"""

##### We have seen this #####
nums = [1,8,9,0]

# for i in nums:
#     print(i)      # This iterates over the list and prints the values....

##### Alternatively we can also use an iterator object #####

itt = iter (nums)
print(itt) ##### It prints the object itself

##### want get the values #####
print(itt.__next__()) ##### here we do not have to use the index values
print(itt.__next__()) ### next will always give you the next value. It always
   ↳ knows the last value and return the next.
print(next(itt)) ### this also works..
```

```
[ ]: ##### Can we have our own iterator object #####
"""
1. Previous iterator object is made and returned by the in-built function i.e.
   ↳ iter().
2. Can we make our own iterator object? Yes for that, we need to design our own
   ↳ class.
3. A python iterator object must implement two special methods, __iter__() and
   ↳ __next__().
"""

class Topfive:

    def __init__(self): # Helps in setting the initial value
        self.num = 1

    def __iter__(self): # return the iterator object. Helps to make the object
   ↳ of the class, behave like an iterator
        return self

    def __next__(self): # Next method returns the next item in the sequence. On
   ↳ reaching end, it must raise StopIteration
        if self.num <= 5:
```

```

        val = self.num
        self.num += 1
        return val
    else:
        raise StopIteration ##### raise exception to stop the for loop

values = Topfive() ##### Once the object of topfive is created, it behaves like
↳ an iterator.

print(values.__next__()) ##### without index, you can get the next values in
↳ sequence.

# for i in values:
#     print(i)

```

```

[ ]: ##### Generator in python #####
"""
1. So in iterators, we need to define two functions __iter__() and __next__().
↳ If you want Python to do all for us,
you can use generators.
2. The generators will give you the iterators.
3. How can we use generators?
"""

# def topfive():#Converting this function into a generator by a special keyword
↳ called yield, normally we have return in func.
#     yield 5    # Generators primarily have yield instead of return

# val = topfive() # So topfive is a generator function which is returning the
↳ iterator object

# print(val.__next__()) ## So on iterator __next__() will work.
# print(val.__next__()) # one value is there in the generator----> so throwing
↳ an exception.

# def topfive():#Converting this function into a generator by a special keyword
↳ called yield, normally we have return in func.
#     yield 5
#     yield 6

# val = topfive()

# print(val.__next__())
# print(val.__next__()) # no error now. For loop can also be used to fetch the
↳ values from iterator.

```

```

##### How to create generator that returns multiple values
↳ #####
def topfive(): # Generator function having a yield

    n = 1

    while n <= 5:
        yield n    # Yield does not terminates a function like return does. It
↳ will continuously send values.
        n+=1

values = topfive()

for i in values:
    print(i)

```

```

[ ]: ##### Imperative vs Declarative programming #####
"""
1. Imperative Programming as the name suggests is a type of programming
↳ paradigm that describes how the program executes.
Developers are more concerned with how to get an answer step by step. This
↳ includes procedural programming (Procedural: Tasks are treated as
↳ step-by-step
iterations where common tasks are placed in functions that are called as needed.
↳ ), object-oriented programming,
parallel processing approach.

2. Declarative Programming as the name suggests is a type of programming
↳ paradigm that describes what programs to be
executed. Developers are more concerned with the answer that is received. This
↳ includes logic programming and functional
programming.
"""

##### Example of Imperative and Declarative programming #####

# Imperative approach to find the sum of a list

# l1 = [1,33,4,5]
# total = 0
# for item in l1:
#     total += item
# print(total)

```

```
# Declarative approach to find the sum of a list
```

```
l1 = [1,33,4,5]
```

```
print(sum(item for item in l1))
```

```
#print(sum(l1))
```